

Collaborative Knowledge Marketplace

Project Report — Production Feature & Architecture Overview

1. Project Overview

- Collaborative Knowledge Marketplace (CKM) is a MERN-based knowledge and learning platform connecting learners, creators, experts, and administrators.
- Core objective: provide course learning, resources, progress tracking, mentorship, reviews, and platform administration using real backend data.

2. Technology Stack

- Frontend: React 19, Vite, React Router DOM, Tailwind CSS, Framer Motion, Lucide React, Context API.
- Backend: Node.js, Express.js, MongoDB, Mongoose, JWT, Passport.js, Google OAuth, Socket.IO, Nodemailer.
- Storage/Deployment: MongoDB Atlas, Cloudinary, Multer, Vercel frontend, Render backend.

3. Authentication & Security

- Email/password authentication with HTTP-only cookie-based JWT authentication.
- Google OAuth through Passport.js.
- Two-factor authentication using email OTP for applicable users.
- Email verification and password-reset flows.
- Role-based authorization for learner, creator, expert, and admin.
- Frontend authorization complements, but never replaces, backend authorization.

4. User & Profile System

- User accounts contain role and authentication information.
- Separate role profiles support learner, creator, and expert-specific information.
- Admins can manage platform users and relevant role profiles.

5. Category Module

- Categories are managed through backend CRUD functionality.
- Category access and management follow the existing role permissions.
- Categories are used for organizing and filtering courses/resources.

6. Resource Management

- Resources support title, description, category, topics, documents, links, thumbnail, and status.
- Documents and media use the existing Multer/Cloudinary flow.
- Creators and experts manage their own resources according to authorization rules.
- Users can explore and view resources and use filtering/search where supported.

7. Course Management

- Courses contain title, description, thumbnail, category, topics, creator, resources, and status.
- Creators/experts can manage courses and organize learning content.
- Course lifecycle supports draft, published, and archived states where provided by the backend.

8. Unit & Learning Content

- Courses are organized into units.
- Units can contain resources and playable YouTube videos.
- Authorized creators/experts can create and manage units and their learning content.

9. Course Enrollment

- Learners can enroll in available courses and access their enrolled courses.
- Enrollment state is synchronized between course details, exploration, My Courses, and the learner dashboard.
- Enrolled courses show an appropriate enrolled/continue-learning state instead of an incorrect enrollment action.

10. Learning Progress

- Progress is based on actual unit completion records.
- Course progress is derived from real progress data and available unit totals.
- Unit completion updates the learner's course progress and dashboard state.

11. Bookmark System

- Learners can bookmark supported resources/attachments.
- Bookmarks are displayed through the bookmarks area and learner dashboard.
- Bookmark changes update the relevant UI and backend state without unnecessary full-page reloads.

12. Reviews & Ratings

- Learners can submit reviews for applicable courses.
- Course views/cards use real average ratings and review counts when provided.
- Learner dashboards display the authenticated learner's actual reviews.

13. Mentorship System

- Experts can create and manage mentorship sessions.
- Learners can discover available sessions and submit requests.
- Experts can accept or reject learner requests using authorized backend operations.
- Session states include pending, accepted, completed, rejected, or cancelled where supported.
- Meeting information is exposed only when backend authorization allows access.

14. Learner Dashboard

- Shows real enrolled courses, in-progress/completed courses, learning progress, bookmarks, reviews, and mentorship sessions.
- Includes continue-learning actions and role-appropriate quick actions.
- All statistics are calculated from actual backend data; missing data produces proper empty states.

15. Creator Dashboard

- Shows real course/resource statistics and creator-owned content.
- Supports course management, unit management, learner visibility, resource management, and supported actions.
- Provides quick actions such as creating courses and resources.

16. Expert Dashboard

- Combines course/resource management with mentorship management.
- Shows expert profile information, courses, resources, session statistics, learner requests, and upcoming sessions.
- Supports accepting/rejecting requests and managing session state according to backend permissions.

17. Admin Dashboard

- Provides platform-level statistics using real users, courses, resources, categories, enrollments, and other available backend metrics.
- Supports user, course, resource, category, creator, expert, and report/moderation management where corresponding APIs exist.
- Unavailable backend metrics are not replaced with fake data.

18. Role-Based Navigation

- Learner: Dashboard, Explore Courses, My Courses, Resources, Bookmarks, Sessions, Reviews, Profile.
- Creator: Dashboard, Courses, Resources, Categories, Profile.
- Expert: Dashboard, Courses, Resources, Sessions, Manage Learners, Profile.
- Admin: Dashboard, Users, Courses, Resources, Categories, Creators, Experts, Reports, Settings.
- Only routes that actually exist are exposed.

19. Dashboard Data Integrity

- No hardcoded statistics, fake users, mock courses, dummy sessions, placeholder reviews, fake progress, or randomly generated dashboard values.
- Every displayed value comes from backend responses, authenticated-user data, or calculations derived from real backend data.
- Loading, error, and empty states are handled explicitly.

20. Data Synchronization

- Enrollment, progress, bookmarks, reviews, and mentorship state changes are reflected across related pages.
- Existing frontend state/query mechanisms should be reused to avoid unnecessary duplicate requests.
- Mutations should update or invalidate affected data rather than requiring unnecessary full-page refreshes.

21. UI/UX & Motion

- Responsive production-style interface for desktop, tablet, and mobile.
- Existing visual identity uses dark-blue/glass-inspired surfaces with a light theme option.
- Framer Motion is used for page entrances, staggered lists, card interactions, buttons, navigation, dialogs, tabs, progress, enrollment/bookmark state changes, and meaningful session transitions.
- Animations remain short and intentional and respect prefers-reduced-motion.

22. Backend Architecture

- Express routes/controllers/services/models remain modular and follow existing project conventions.
- Existing API clients/services and authentication configuration are reused rather than duplicated.
- Required missing APIs should receive only the smallest compatible backend change.

23. File & Media Handling

- Uploaded documents/media use the existing Multer and Cloudinary pipeline.

- Frontend does not hardcode backend URLs inside individual components; existing environment/API configuration is used.

24. Deployment

- Frontend: Vercel.
- Backend: Render.
- Database: MongoDB Atlas.
- Production authentication requires correct CORS credentials and secure cross-site HTTP-only cookies.
- Google OAuth callback and frontend SPA routing must use the deployed application URLs.

25. Core User Journeys

- Learner: Login → Dashboard → Explore → Enroll → My Courses → Unit → Complete Unit → Progress → Bookmark → Review → Request Mentorship → Acceptance → Authorized Meeting Access → Completion.
- Creator: Login → Dashboard → Create Course → Add Units/Resources → Publish → Manage Learners/Content.
- Expert: Login → Dashboard → Profile → Course/Resource Management → Create Session → Manage Requests → Accept/Reject → Complete Session.
- Admin: Login → Dashboard → Platform Statistics → Users → Courses → Resources → Categories → Experts/Creators → Reports.

26. Production Principles

- Correct data over visual completeness.
- Backend authorization over frontend-only hiding.
- Real-time/state synchronization over stale UI.
- Reusable components over duplicated logic.
- Existing architecture over unnecessary restructuring.
- No demo data or temporary mock implementations in production code.